

MATH3881/5881: Statistical Machine Learning Theory

Week 7: Sequence based models

Sahani Pathiraja

UNSW, Sydney

Key Topics

7.1	Dealing with sequences of data	7-1
7.2	Recurrent Neural Networks (RNNs)	7-2
7.3	RNN training: Backpropagation through time	7-5
7.4	Universal approximation capability	7-13
7.5	RNN Implementation Aspects	7-14
7.6	Long Short Term Memory Networks (LSTMs)	7-15
7.7	Backpropagation through time for LSTMs	7-18
7.8	How do LSTMs mitigate vanishing gradients?	7-21
7.9	LSTM implementation aspects and limitations	7-22
7.10	Tutorial exercises	7-24
	References	7-26
A	[Non-examinable] Proof of Universal Approximation Theorem	7-26

7.1 Dealing with sequences of data

Up until now, we have assumed that our training data is independent and identically distributed. What happens when we want to fit models to data that have a sequential structure? Take for example a sentence; each word in a sentence has a natural link to words preceeding it, i.e. there are correlations. The data is *ordered*, meaning that you can't reshuffle the dataset and have the same learning problem. In contrast, many standard classification problems can be treated as depending only on the collection of features present rather than their ordering. For example, a bag-of-words text classifier would treat the sentences **machine learning is fun** and **fun learning machine is** identically because both contain exactly the same words. For sequence modelling, however, the order carries essential information and changing that order fundamentally changes the prediction problem. The same is true for audio sequences, many types of financial (e.g. stock price) and environmental (e.g. rainfall) data and arguably also sequences of images (i.e. videos).

Processing such kinds of data streams has traditionally been the domain of *time series analysis*. You may have heard of Markov models, Hidden Markov Models, ARIMA, etc., which are widely used in forecasting and prediction in economics, finance, environment, etc. Many of these models are *highly structured*, in that they require the data to obey certain assumptions. For example, the data maybe assumed to come from a model satisfying the Markov property. In this lecture, we will be focusing on modern machine learning methods which instead try to learn the temporal structure from the data itself. We'll focus on two major sequence based models: **Recurrent Neural Networks**

and **Long Short Term Memory models**. Although state of the art sequence modelling is now dominated by *Transformers*, the methods we will see in this lecture are still useful in low dimensional problems with moderate data sizes, and in understanding the evolution of sequence modelling that led to Transformers.

Note that sequence based data need not depend on time only. It is possible to think of spatial data as also being sequence based. For example, an image can be partitioned into a collection of small patches. These patches can then be arranged into a sequence according to their spatial locations (e.g. scanning row-by-row across the image). Clearly the ordering here is important, as shuffling the locations of the patches leads to a completely different image.

Generally, the dataset of interest consists of N sequences, taking the form

$$\mathcal{D} = (x_1, y_1), \dots, (x_N, y_N),$$

where x_i is the input, y_i the output, the i th sequence has length T_i and

$$x_i = (x_i^{(1)}, \dots, x_i^{(T_i)}).$$

Importantly, the length of each sequence T_i need not be the same, and this is one of the strengths of the methods we will see. Some typical sequence based tasks are:

- **sequence to sequence prediction.** e.g. a sentence in english is mapped to its corresponding sentence in french.
- **Sequence classification.** A model analyses an entire sequence and assigns it a single label or category. Examples include sentiment analysis of a sentence (positive/negative) or classifying an ECG signal as normal or abnormal.
- **Next value prediction.** A model uses previous values in a sequence to predict the next value. Examples include forecasting tomorrow's stock price or the next word in a sentence.

7.1.1 Some Notation

Unlike Weeks 2 to 5, we will use ϕ instead of σ to indicate a general non-linear activation function (unless otherwise specified). This is primarily as we will reserve the use of σ for standard deviation as in the usual statistics convention.

For norms, we use absolute value $|\cdot|$ for scalar quantities, the Euclidean norm $\|\cdot\|_2$ for vectors, and the induced spectral norm $\|\cdot\|_2$ for matrices.

7.2 Recurrent Neural Networks (RNNs)

RNNs are most easily explained by appealing to a concrete prediction task: Given a partial segment of a sentence, what is the most likely word or sequence of words that complete the sentence? Example 7.2 will be helpful in understanding the structure of RNNs.

Example 7.2.1

Consider a simple lexicon (collection of words), consisting of

$$\{\text{machine, best, is, learning, the, fun}\}$$

where each word is represented by a unit vector in $\mathbb{R}^6$ (known as one-hot-encoding). For example, the word “is” is represented by $[0, 0, 1, 0, 0, 0]$. We are given a partial sentence, which we denote by $x = \{x^{(1)}, x^{(2)}, \dots, x^{(d)}\}$, where d denotes the length of the sequence. Suppose we receive the text **machine learning is**, then $d = 3$ and $x^{(1)} = [1, 0, 0, 0, 0, 0]$ and $x^{(2)} = [0, 0, 0, 1, 0, 0]$ and so on. We want to know what is the next word in the sentence. RNNs can be used to produce an output vector $\hat{y}^{(4)}$, where $\hat{y}^{(4)} \in \mathbb{R}^6$ is a unit vector representing the probability mass function of the next word. For example, it may yield

$$\hat{y}^{(4)} = [0, 0.1, 0, 0, 0.1, 0.8]$$

A maximum likelihood based decision would then select **fun** as the next word.

At an intuitive level, RNNs can be seen as feedforward neural networks with *cyclical connections*. Before unpacking this viewpoint, we’ll take a look at the form of a simple RNN and its building blocks. As with CNNs and FNNs, an *input* (i.e. the partial sentence in the previous example) is provided to the RNN and it produces an *output*. In the below description we’ll drop the dependence on i (the element of the dataset) for notational clarity. In a simple RNN, this transformation occurs via

$$h^{(t)} = \phi(Wh^{(t-1)} + Ux^{(t)} + b), \quad (7.1)$$

$$\hat{y}^{(t)} = f_y(Vh^{(t)} + c) \quad (7.2)$$

for $t = 1, 2, 3, \dots, T$. The function $\phi(x) : \mathbb{R}^m \rightarrow \mathbb{R}^m$ is a nonlinear activation function, $x^{(t)} \in \mathbb{R}^d$ is an input at time t , $h^{(t)} \in \mathbb{R}^m$ is a hidden state vector (or hidden units) at time t , $\hat{y}^{(t)} \in \mathbb{R}^n$ is the output probability vector and W, V, U, b, c are trainable parameters of the RNN; $W, V \in \mathbb{R}^{m \times m}$, $U \in \mathbb{R}^{m \times d}$ being the weight matrices and $b, c \in \mathbb{R}^m$ being the bias vectors. In this lecture, the hidden states are assumed to lie in the Euclidean space $\mathbb{R}^m$ ¹. Typical choices for f_y include the softmax function (for multi-class classification and language modelling), the sigmoid function (for binary classification), or simply the identity map (for regression). When probabilistic outputs are required, the softmax function is commonly used because it converts arbitrary real-valued scores into a probability distribution whose entries are non-negative and sum to one. The affine transformation $Vh^{(t)} + c$ maps the hidden state into an output space appropriate for the task at hand. In language modelling, for example, the hidden state may have dimension m while the output space has dimension equal to the vocabulary size. The matrix V therefore acts as a readout or observation operator, converting information stored in the hidden state into quantities from which predictions can be made.

Equations (7.1)–(7.2) are commonly known as the *Elman equations* [1]. The key idea is that the hidden state is recursively updated using both the current input and the previous hidden state,

¹More specialised neural architectures have also been developed for non-Euclidean domains such as graphs, manifolds and hyperbolic spaces, but these lie beyond the scope of this course.

thereby allowing information from earlier parts of the sequence to influence future predictions. Notice in particular that the term $Wh^{(t-1)}$ is what distinguishes (7.1) from a FNN, and what gives rise to *cyclical connections*, see also the circular arrow in Figure 7.1 (left). Figure 7.1 shows the concept of *unfolding* in ML terminology. Mathematically, this just refers to the fact that the recursive process (7.1) can be algebraically reformulated, using the shorthand notation $f(h, x) := \phi(Wh + Ux + b)$ as

$$\begin{aligned} h^{(1)} &= f(h^{(0)}, x^{(1)}) \\ h^{(2)} &= f(h^{(1)}, x^{(2)}), \\ &= f(f(h^{(0)}, x^{(1)}), x^{(2)}) \\ h^{(3)} &= f(h^{(2)}, x^{(3)}), \\ &= f(f(f(h^{(0)}, x^{(1)}), x^{(2)}), x^{(3)}) \\ &\vdots \end{aligned}$$

so that $h^{(t)}$ at any time t can be written entirely in terms of $h^{(0)}$ and the sequence of inputs x . This algebraic reformulation is useful in identifying the *width* and *depth* of a simple RNN as well as their connections to FNNs. Notice that once unfolded, a simple RNN has the same structure as a FNN with multiple layers.

The hidden state $h^{(t)}$ can be interpreted as the RNN's internal memory of everything it has observed up to time t . The dimension m of the hidden state therefore places a limit on how much information can be retained: larger hidden states provide greater representational capacity and allow the network to encode more complex temporal dependencies, albeit at the cost of additional parameters and computation.

Remark 7.2.1

The concept of *depth* in a RNN is not entirely well-defined and depends on context. When a simple RNN is unfolded through time, it is common to interpret the depth as the number of recurrent updates, i.e. the time horizon T . However, in architectures such as *stacked RNNs*^a, depth usually refers to the number of recurrent layers that are composed together at each time step. It is often useful to distinguish between *temporal depth* (the length of the sequence) and *architectural depth* (the number of stacked recurrent layers).

^aout of scope of this course

Exercise 7.2.1

Consider the simple RNN given in Figure 7.1, along with (7.1)-(7.2). What is the depth and width?

Exercise 7.2.2

Why do RNNs have cyclical connections (i.e. a recurrence relation) in their hidden state, unlike FNNs that have only forward information flow?

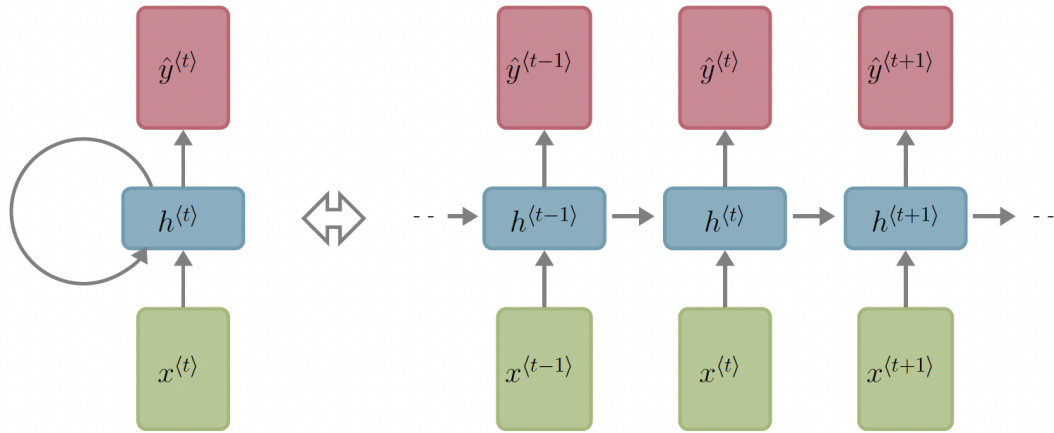


Figure 7.1: Schematic of a simple RNN

7.3 RNN training: Backpropagation through time

RNNs are most often trained through a technique known as *Backpropagation through time (BPTT)*. As the name suggests, this is very similar to the Backpropagation algorithm you have already seen in Week 2, except that there is now an extra element involving gradients through time. Once these gradients are evaluated, they can be used in a gradient based optimiser such as stochastic gradient descent or Adam. Given an RNN with depth $T = \max_i T_i$, the training loss function is comprised of the sum of the chosen loss function over each element and over all sequences,

$$\mathcal{L} = \sum_{i=1}^N \sum_{t=1}^{T_i} \ell_i^{(t)}, \quad \ell_i^{(t)} := \ell(\hat{y}_i^{(t)}, y_i^{(t)}) \quad (7.3)$$

where $\hat{y}_i^{(t)}$ is the output of the RNN and $y_i^{(t)}$ is the observed data at the time index t in the i th sequence and ℓ is a chosen loss function, e.g. cross-entropy loss for a classification problem.

Important!

For the rest of this section, we will assume $N = 1$ and drop the dependence on the i th data point to reduce notational overhead.

The initial hidden state $h^{(0)}$ is usually set to the zero vector, although it can also be learned as a parameter in some implementations. We'll focus on demonstrating BPTT for a simple RNN. Recall that an RNN can be algebraically reformulated, via *unfolding* or *unrolling* to an FNN with T layers, where the parameters are shared across the layers. This makes it straightforward to apply Backpropagation, with the important difference to the usual FNN training being that now each “layer” has the same parameters. This is the defining feature of BPTT: because the parameters are shared across time, their gradients are also accumulated across time. We will see this more precisely below, following similar notation to Backpropagation in Week 2.

We'll introduce some shorthand notation to help with evaluating the relevant derivatives. Consider the simple RNN in (7.1)-(7.2). Define also,

$$z^{(t)} := Vh^{(t)} + c, \quad (7.4)$$

$$\ell^{(t)} := \ell(y^{(t)}, \hat{y}^{(t)}), \quad (7.5)$$

so then the simple RNN is written as

$$h^{(t)} = \phi(Wh^{(t-1)} + Ux^{(t)} + b)$$

$$\hat{y}^{(t)} = f_y(z^{(t)}),$$

We'll also rely on the following important assumption (as assumed in Week 2):

Assumption 7.3.1

The nonlinear activation function $\phi : \mathbb{R}^m \rightarrow \mathbb{R}^m$ acts componentwise and we use the abuse of notation $\phi(a) = [\phi(a_1), \phi(a_2), \dots, \phi(a_m)]^\top$

7.3.1 Detour: Refresher on Multivariate calculus

Another important clarification point will be *partial* vs *total* derivatives, as distinguishing the two will be much more important here than in our discussion of Backpropagation in Week 2. Let $f(x(t), y(t))$ be a differentiable scalar valued function with $x, y, t \in \mathbb{R}$. The total derivative of f with respect to t is

$$\frac{df}{dt} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

obtained via chain rule. Recall that $\frac{\partial f}{\partial x}$ means differentiate f with respect to x while keeping all other variables fixed. In this example, $\frac{\partial f}{\partial t}$ is not well defined since f depends on t through other variables x, y , so the total derivative is the relevant gradient quantity here. However, this is not strictly adhered to in the ML literature so that you may often find ∂ and d used interchangeably. Whenever a variable depends only on one other variable, the ∂ and d notation are sometimes used interchangeably in ML literature, although here we will (mostly) use d . However, as we are dealing with vector-valued variables, we may also adopt the ∇ notation. Recall that for $f(x) : \mathbb{R}^d \rightarrow \mathbb{R}$,

$$\nabla_x f = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_d} \right]^\top \in \mathbb{R}^{d \times 1}$$

which then becomes a Jacobian matrix of size $d \times m$ if $f(x) : \mathbb{R}^d \rightarrow \mathbb{R}^m$.

Recall also the chain rule for vector valued functions $z = g(y), y = f(x)$ where $g : \mathbb{R}^m \rightarrow \mathbb{R}^p$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ are both smooth continuous functions. Then

$$\frac{\partial z}{\partial x} = \frac{\partial y}{\partial x} \frac{\partial z}{\partial y} \in \mathbb{R}^{n \times p}$$

with the Jacobian convention² being $\frac{\partial z}{\partial y} \in \mathbb{R}^{m \times p}$ and $\frac{\partial y}{\partial x} \in \mathbb{R}^{n \times m}$. We will use this convention throughout the lecture.

²The Jacobian is sometimes defined as the transpose of what is assumed here.

7.3.2 Gradients for BPTT

In the below, we'll use $\delta_q^{(t)}$ to denote the partial derivative of the loss with respect to a variable q . This quantity is sometimes referred to as the “error signal.”

Output layer gradients. Define the output error signal and using the multivariable chain rule,

$$\delta_z^{(t)} := \frac{\partial \ell^{(t)}}{\partial z^{(t)}} = \frac{\partial \hat{y}^{(t)}}{\partial z^{(t)}} \frac{\partial \ell^{(t)}}{\partial \hat{y}^{(t)}} \in \mathbb{R}^{m \times 1}$$

Then the gradients with respect to the output parameters are

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial c} &= \sum_{t=1}^T \delta_z^{(t)} \\ \frac{\partial \mathcal{L}}{\partial V_{ij}} &= \sum_{t=1}^T \frac{\partial z^{(t)}}{\partial V_{ij}} \delta_z^{(t)} = \sum_{t=1}^T h_j^{(t)} \delta_z^{(t)}, \quad i, j = 1, \dots, m \end{aligned}$$

Hidden layer derivatives. The derivatives with respect to the hidden layer parameters require some more care since they are shared across all recurrent layers. We'll focus on deriving the procedure for the derivative with respect to W , as the same applies for derivatives with respect to b, U . Define the activation error and applying multivariate chain rule,

$$\delta_h^{(t)} := \frac{\partial \ell^{(t)}}{\partial h^{(t)}} = \frac{\partial z^{(t)}}{\partial h^{(t)}} \frac{\partial \ell^{(t)}}{\partial z^{(t)}} = V^\top \delta_z^{(t)} \in \mathbb{R}^{m \times 1}. \quad (7.6)$$

From now on, we will use the shorthand notation

$$a^{(t)} := Wh^{(t-1)} + Ux^{(t)} + b$$

Exercise 7.3.1

What are the dimensions of $\frac{\partial \hat{y}^{(t)}}{\partial z^{(t)}}$, $\frac{\partial \ell^{(t)}}{\partial \hat{y}^{(t)}}$ and ?

Exercise 7.3.2

Using the total derivative formula^a, show that the gradient with respect to the hidden layer weight is given by

$$\frac{dh^{(t)}}{dW_{ij}} = \sum_{k=1}^t \frac{dh^{(k)}}{dW_{ij}} \frac{\partial h^{(t)}}{\partial h^{(k)}}, \quad i, j = 1, \dots, m$$

where $\frac{\partial h^{(t)}}{\partial h^{(k)}} \in \mathbb{R}^{m \times m}$.

^awe stress the use of d over ∂ here even though the derivative is taken with other entries of W held constant, in order to distinguish the two kinds of derivatives.

Then using Exercise 7.3.2, we have that

$$\frac{d\mathcal{L}}{dW_{ij}} = \sum_{l=1}^T \frac{dh^{(l)}}{dW_{ij}} \delta_h^{(l)} = \sum_{l=1}^T \left(\sum_{k=1}^l \frac{dh^{(k)}}{dW_{ij}} \frac{\partial h^{(l)}}{\partial h^{(k)}} \right) \delta_h^{(l)}, \quad (7.7)$$

From a computational point of view, (7.7) works backwards in time just as backpropagation in a FFN works backwards through layers. To see this, we'll rewrite the weight derivative by rearranging terms in (7.7),

$$\frac{d\mathcal{L}}{dW_{ij}} = \sum_{k=1}^T \sum_{l=k}^T \frac{dh^{(k)}}{dW_{ij}} \frac{\partial h^{(l)}}{\partial h^{(k)}} \delta_h^{(l)} \quad (7.8)$$

$$= \sum_{k=1}^T \frac{dh^{(k)}}{dW_{ij}} \underbrace{\left(\sum_{l=k}^T \frac{\partial h^{(l)}}{\partial h^{(k)}} \delta_h^{(l)} \right)}_{=:\Delta_h^{(k)}}, \quad (7.9)$$

where the term $\Delta_h^{(k)} \in \mathbb{R}^{m \times 1}$ is a combination of the local derivative at the current time and all future loss gradients. Notice that $\Delta_h^{(k)} = \frac{\partial \mathcal{L}}{\partial h^{(k)}}$ in the case of a single data point. Now separate the first term in the sum to obtain

$$\Delta_h^{(k)} = \underbrace{\delta_h^{(k)}}_{=\text{local term}} + \underbrace{\sum_{l=k+1}^T \left(\frac{\partial h^{(l)}}{\partial h^{(k)}} \right) \delta_h^{(l)}}_{\text{future contribution}}. \quad (7.10)$$

Now for $l > k$, the chain rule implies

$$\frac{\partial h^{(l)}}{\partial h^{(k)}} = \frac{\partial h^{(k+1)}}{\partial h^{(k)}} \frac{\partial h^{(l)}}{\partial h^{(k+1)}}. \quad (7.11)$$

Substituting into the previous expression yields

$$\begin{aligned} \Delta_h^{(k)} &= \delta_h^{(k)} + \left(\frac{\partial h^{(k+1)}}{\partial h^{(k)}} \right) \sum_{l=k+1}^T \left(\frac{\partial h^{(l)}}{\partial h^{(k+1)}} \right) \delta_h^{(l)} \\ &= \delta_h^{(k)} + \left(\frac{\partial h^{(k+1)}}{\partial h^{(k)}} \right) \Delta_h^{(k+1)} \end{aligned} \quad (7.12)$$

with $\Delta_h^{(T)} = \delta_h^{(T)}$, revealing a recursive structure. This procedure is usually executed by starting at time T and then working backwards using (7.12). Finally, we have due to assumption 7.3 that

$$\frac{\partial h^{(k)}}{\partial W_{ij}} = \dot{\phi}(a_i^{(k)}) h_j^{(k-1)} e_i \in \mathbb{R}^{m \times 1}$$

where e_i is the zero vector with 1 in the i th position. Substituting into (7.9) yields

$$\frac{d\mathcal{L}}{dW_{ij}} = \sum_{k=1}^T \dot{\phi}(a_i^{(k)}) h_j^{(k-1)} (\Delta_h^{(k)})_i$$

where $(\Delta_h^{(k)})_i$ is the i th component of $\Delta_h^{(k)}$. In practice, one often defines the local pre-activation error by substituting (7.12) and using (7.6) and that $\frac{\partial h^{(k+1)}}{\partial h^{(k)}} = W^\top \text{diag}(\dot{\phi}(a^{(k)}))$

$$\Delta_a^{(k)} := \frac{\partial \mathcal{L}}{\partial a^{(k)}} = \Delta_h^{(k)} \odot \dot{\phi}(a^{(k)}) \quad (7.13)$$

$$= \left(\delta_h^{(k)} + \left(\frac{\partial h^{(k+1)}}{\partial h^{(k)}} \right) \Delta_h^{(k+1)} \right) \odot \dot{\phi}(a^{(k)}) \quad (7.14)$$

$$= \left(\delta_h^{(k)} + \underbrace{W^\top \text{diag}(\dot{\phi}(a^{(k)})) \Delta_h^{(k+1)}}_{=\Delta_a^{(k+1)}} \right) \odot \dot{\phi}(a^{(k)}) \quad (7.15)$$

$$= \left(V^\top \delta_z^{(k)} + W^\top \Delta_a^{(k+1)} \right) \odot \dot{\phi}(a^{(k)}) \quad (7.16)$$

where $\odot$ denotes the componentwise product (sometimes also referred to as Schur or Hadamard product), which again yields a recursion in $\Delta_a^{(k)}$, so then the final derivative is given by

$$\frac{d\mathcal{L}}{dW_{ij}} = \sum_{k=1}^T h_j^{(k-1)} (\Delta_a^{(k)})_i \quad (7.17)$$

where the sum is evaluated **backwards** from T and usually $\Delta_a^{(T+1)} = 0$ to initialise.

Exercise 7.3.3

It is known that (7.7) is mathematically correct but computationally inefficient. Why is this the case?

Exercise 7.3.4

Gradients with respect to U and b . Derive the derivatives of $\mathcal{L}$ with respect to the remaining hidden parameters, U_{ij} and b using a similar strategy presented for the case of W . The same backward-recursion idea applies to the input weight matrix U and the bias vector b . Since the parameters U and b are shared across all time steps, their derivatives are also accumulated over time.

The final backpropagation algorithm is shown in Algorithm 1.

Algorithm 1 Backpropagation Through Time for a Simple RNN

Require: Input sequence $x^{(1)}, \dots, x^{(T)}$, targets $y^{(1)}, \dots, y^{(T)}$, parameters W, U, b, V, c, T

- 1: Initialize $h^{(0)} \leftarrow 0$
 - 2: Initialize $\nabla_W \mathcal{L} \leftarrow 0, \nabla_U \mathcal{L} \leftarrow 0, \nabla_b \mathcal{L} \leftarrow 0, \nabla_V \mathcal{L} \leftarrow 0, \nabla_c \mathcal{L} \leftarrow 0$ ▷ Forward pass
 - 3: **for** $t = 1$ to T **do**
 - 4: $a^{(t)} \leftarrow Wh^{(t-1)} + Ux^{(t)} + b$
 - 5: $h^{(t)} \leftarrow \phi(a^{(t)})$
 - 6: $z^{(t)} \leftarrow Vh^{(t)} + c$
 - 7: $\hat{y}^{(t)} \leftarrow f_y(z^{(t)})$
 - 8: $\ell^{(t)} \leftarrow \ell(y^{(t)}, \hat{y}^{(t)})$
 - 9: $\delta_z^{(t)} \leftarrow \nabla_{z^{(t)}} \ell^{(t)}$
 - 10: **end for**
 - 11: $\mathcal{L} \leftarrow \sum_{t=1}^T \ell^{(t)}$ ▷ Backward pass
 - 12: $\Delta_a^{(T+1)} \leftarrow 0$
 - 13: **for** $t = T$ down to 1 **do**
 - 14: $\Delta_h^{(t)} \leftarrow \underbrace{V^\top \delta_z^{(t)}}_{=\delta_h^{(t)}} + W^\top \Delta_a^{(t+1)}$
 - 15: $\Delta_a^{(t)} \leftarrow \Delta_h^{(t)} \odot \dot{\phi}(a^{(t)})$
 - 16: $\nabla_W \mathcal{L} \leftarrow \nabla_W \mathcal{L} + \Delta_a^{(t)} (h^{(t-1)})^\top$
 - 17: $\nabla_U \mathcal{L} \leftarrow \nabla_U \mathcal{L} + \Delta_a^{(t)} (x^{(t)})^\top$
 - 18: $\nabla_b \mathcal{L} \leftarrow \nabla_b \mathcal{L} + \Delta_a^{(t)}$
 - 19: $\nabla_V \mathcal{L} \leftarrow \nabla_V \mathcal{L} + \delta_z^{(t)} (h^{(t)})^\top$
 - 20: $\nabla_c \mathcal{L} \leftarrow \nabla_c \mathcal{L} + \delta_z^{(t)}$
 - 21: **end for**
 - 22: **return** $\nabla_W \mathcal{L}, \nabla_U \mathcal{L}, \nabla_b \mathcal{L}, \nabla_V \mathcal{L}, \nabla_c \mathcal{L}$
-

Algorithm 2 One training epoch for a simple RNN

Require: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, parameters W, U, b, V, c, T , learning rate $\eta > 0$ 1: **for** $i = 1, \dots, N$ **do**2: For objective function $\mathcal{L}_i = \sum_{t=1}^T \ell_i^{(t)}$, compute

$$\nabla_W \mathcal{L}_i, \nabla_U \mathcal{L}_i, \nabla_b \mathcal{L}_i, \nabla_V \mathcal{L}_i, \nabla_c \mathcal{L}_i$$

by applying Algorithm 1 to $(x_i^{(t)}, y_i^{(t)})_{t=1, \dots, T}$ using the current parameters.

3: Update the parameters:

$$W \leftarrow W - \eta \nabla_W \mathcal{L}_i,$$

$$U \leftarrow U - \eta \nabla_U \mathcal{L}_i,$$

$$b \leftarrow b - \eta \nabla_b \mathcal{L}_i,$$

$$V \leftarrow V - \eta \nabla_V \mathcal{L}_i,$$

$$c \leftarrow c - \eta \nabla_c \mathcal{L}_i.$$

4: **end for**5: **return** W, U, b, V, c

Note that the same structure in Algorithm 2 applies when using an adaptive optimiser such as Adam.

7.3.3 Vanishing and exploding gradients

The vanishing gradient phenomenon that you saw in Week 2 is also an issue in training RNNs via gradient descent. It is primarily caused by a poorly conditioned optimisation problem (e.g. nearly flat loss landscape around the optimum), which then makes gradient descent based methods prohibitively inefficient. BPTT is sometimes mistakenly cited as the cause of the vanishing gradient phenomenon; it is important to note this is not true. In fact, BPTT actually helps to clarify why it occurs, as it is not immediately clear from the simple gradient descent formula.

The vanishing gradient problem is particularly damaging in the context of RNNs because it prevents the network from learning long-range dependencies. Parameters that affect early parts of a sequence receive little gradient information and therefore update very slowly. As a result, a simple RNN may struggle to learn relationships between events that are separated by many timesteps. For example, to predict the next word (with high probability) in the sentence **the clouds are in the** , we don't need to know much about the words before the sentence (there is a high chance the next word is **sky**). However, the next word in the sentence **I speak fluent** is much less predictable, as it could be any language. However, if the phrase **I grew up in France** appears much earlier on in a text, that context would help us predict the language as **french**. It is this kind of long-range dependence or *context* that is difficult for simple RNNs to capture. This limitation is one of the main motivations for more sophisticated recurrent architectures such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs).

We will now investigate how it occurs. Returning to (7.9), we see that the summand there involves

computing a product of Jacobian matrices,

$$\frac{\partial h^{(l)}}{\partial h^{(k)}} = \prod_{j=k+1}^l \frac{\partial h^{(j)}}{\partial h^{(j-1)}} = \prod_{j=k+1}^l W^\top \text{diag}(\dot{\phi}(a^{(j)})) \quad (7.18)$$

using the multivariable chain rule and since ϕ applies componentwise. So then,

$$\frac{d\mathcal{L}}{dW_{ij}} = \sum_{l=1}^T \left(\sum_{k=1}^l \left(\frac{dh^{(k)}}{dW_{ij}} \right)^\top \left(\prod_{j=k+1}^l W^\top \text{diag}(\dot{\phi}(a^{(j)})) \right) \right) \delta_h^{(l)}, \quad (7.19)$$

Let $D^{(j)} := \text{diag}(\dot{\phi}(a^{(j)})) \in \mathbb{R}^{m \times m}$. We'll also adopt the shorthand notation,

$$P_{l:k} := \frac{\partial h^{(l)}}{\partial h^{(k)}} = \prod_{j=k+1}^l W^\top D^{(j)},$$

This structure reveals that learning long-range dependence requires multiplying many Jacobian (and weight) matrices. In particular, the term $P_{l:k}$ appears whenever a change at time k influences the loss at a later time l . To understand the impact of this, suppose that for a time interval $k+1, \dots, l$, there exist constants $0 < \rho_D$ and $\rho_W > 0$ such that

$$\|W\|_2 \leq \rho_W, \quad \|D^{(j)}\|_2 \leq \rho_D, \quad \text{for all } j = k+1, \dots, l.$$

Then by submultiplicativity of the matrix norm,

$$\|P_{l:k}\|_2 = \left\| \prod_{j=k+1}^l W^\top D^{(j)} \right\|_2 \leq \prod_{j=k+1}^l \|D^{(j)}\|_2 \|W\|_2 \leq (\rho_D \rho_W)^{l-k}.$$

Therefore, the contribution from time l to the scalar derivative $d\mathcal{L}/dW_{ij}$ can be bounded as

$$\begin{aligned} \left| \sum_{k=1}^l \left(\frac{dh^{(k)}}{dW_{ij}} \right)^\top P_{l:k} \delta_h^{(l)} \right| &\leq \sum_{k=1}^l \left| \left(\frac{\partial h^{(k)}}{\partial W_{ij}} \right)^\top P_{l:k} \delta_h^{(l)} \right| \leq \sum_{k=1}^l \left\| \frac{\partial h^{(k)}}{\partial W_{ij}} \right\|_2 \|P_{l:k}\|_2 \|\delta_h^{(l)}\|_2 \\ &\leq \sum_{k=1}^l \left\| \frac{\partial h^{(k)}}{\partial W_{ij}} \right\|_2 (\rho_D \rho_W)^{l-k} \|\delta_h^{(l)}\|_2. \end{aligned}$$

Now suppose $\rho_D \rho_W < 1$, then each recurrent Jacobian is contractive, i.e. the term $(\rho_D \rho_W)^{l-k}$ decays exponentially in the temporal distance $l-k$. In optimisation terms, this means that perturbations to the hidden state at time k have progressively less influence on the hidden state at later times. The loss function becomes increasingly insensitive to parameters. Thus the optimisation problem becomes poorly conditioned in the directions required to learn long-range dependencies. This vanishing gradient phenomenon makes it difficult to learn long-range dependencies (when $l-k$ is large) via gradient descent.

The opposite phenomenon can also occur, when $\rho_D \rho_W > 1$. In this case the corresponding gradient contributions can become extremely large, producing unstable parameter updates. This is known as the *exploding gradient* phenomenon.

What leads to $\|D^{(j)}\|_2 \leq \rho_D$ with $0 < \rho_D < 1$? For example, if $\phi = \tanh$, then

$$\dot{\phi}(a) = 1 - \tanh^2(a),$$

so

$$0 \leq \dot{\phi}(a) \leq 1.$$

Moreover, when $|a|$ is large, $\tanh(a)$ is close to either 1 or -1 , and therefore $\dot{\phi}(a) \approx 0$, so that $D^{(j)}$ may have very small singular values, i.e. ρ_D is close to zero. On the other hand, $|a|$ increases as $\|W\|_2$ increases (keeping all other variables fixed), so that it is near impossible to ensure $\rho_D \rho_W$ is close to 1.

Remark 7.3.2

It is important to distinguish between representation capability and learnability by gradient descent. The above discussion does not show that RNNs are incapable of representing long-range dependencies. For example, in a simple linear recurrent system,

$$h^{(t)} = Wh^{(t-1)},$$

if $W = I$, then $h^{(t)} = h^{(0)}$ for all t . Therefore, the system can preserve information indefinitely. More generally, suitable recurrent weights can in principle implement long-memory behaviour. In fact, there are known *Universal Approximation Theorems* for RNNs as we will briefly touch on below. The issue is that in many cases, gradient based methods have difficulty learning such weights.

7.4 Universal approximation capability

Similar to FNNs, RNNs possess a universal approximation capability, although the precise meaning of this differs to the FNN case. There are also a few different flavours of “universal approximation” theorems. As with FNNs, there exist theorems that address universal approximation capability in terms of 1) arbitrary width and 2) arbitrary depth. When they say “arbitrary” what is really meant is that there exists a *sufficiently large* dimension of $h^{(t)}$ i.e. number of neurons per layer (for width) or time horizon T (for depth). Results on arbitrary depth UATs are much more recent and their proof strategies are more advanced. We’ll focus on outlining the theorem of Schaffer and Zimmerman [2], which shows that for a fixed *depth*, an RNN with sufficiently large hidden-state dimension can approximate the input-output behaviour of an open dynamical system over any finite time horizon. For simplicity, we will focus on the case of an affine output $z^{(t)}$.

Theorem 7.4.1

UAT for RNNs [2]. Let $x^{(t)} \in \mathbb{R}^d$ be an input at time t , $s^{(t)} \in \mathbb{R}^m$ be a hidden state at time t and $z^{(t)} \in \mathbb{R}^n$ be an output vector at time t and $t \in \{1, 2, \dots, T\}$. Consider the open dynamical system

$$s^{(t)} = g(s^{(t-1)}, x^{(t)}) \quad (7.20)$$

$$z^{(t)} = Qs^{(t)} + r, \quad (7.21)$$

where $g : \mathbb{R}^m \times \mathbb{R}^d \rightarrow \mathbb{R}^m$ is a continuous map and $Q \in \mathbb{R}^{n \times m}$. Consider also the simple RNN

$$h^{(t)} = \phi(Wh^{(t-1)} + Ux^{(t)} + b), \quad (7.22)$$

$$\hat{y}^{(t)} = Vh^{(t)} + c, \quad (7.23)$$

with ϕ the sigmoid activation function. Assume that $s^{(0)} = h^{(0)}$. Then for any finite T , there exists an $\varepsilon > 0$ such that

$$\|z^{(t)} - \hat{y}^{(t)}\|_2 < \varepsilon, \text{ for all } t = 1, 2, \dots, T.$$

Note that Theorem 7.4 is a finite memory/horizon result. This is different from approximating a system uniformly over an infinite time horizon, which generally requires additional stability or fading-memory assumptions. As with feedforward universal approximation theorems, the result is an existence theorem: it does not tell us how large the hidden dimension must be, how to find the parameters efficiently, or how well gradient-based training will perform.

7.5 RNN Implementation Aspects

Although the mathematical formulation of a simple RNN is straightforward, several practical considerations influence both computational efficiency and the ability of the model to learn long-range temporal dependencies. We will very briefly discuss a few of these below, for further reading, see e.g. [3, Chapter 10], [4] and [5, Chapter 7].

- **Leaky units.** A simple way to retain information over longer timescales is to replace the standard hidden-state update by

$$h^{(t)} = \alpha h^{(t-1)} + (1 - \alpha) \tilde{h}^{(t)}, \quad 0 < \alpha < 1.$$

where $\tilde{h}^{(t)} = \phi(Wh^{(t-1)} + Ux^{(t)} + b)$. When α is close to one, information decays more slowly, improving gradient propagation and motivating later gated architectures such as LSTMs and GRUs.

- **Mini-batching.** RNNs are typically trained using mini-batches, with shorter sequences padded to match the longest sequence in the batch. Padding is ignored when computing the loss and gradients, allowing efficient parallel computation on modern hardware.

- **Truncated BPTT.** Full BPTT propagates gradients through the entire sequence, which can be prohibitively expensive for long sequences. Truncated BPTT limits backpropagation to the most recent τ time steps, reducing memory and computation at the cost of potentially missing long-range gradient information.
- **Gradient clipping.** Exploding gradients are commonly mitigated using gradient clipping: if the gradient norm exceeds a threshold c , it is rescaled according to

$$g \leftarrow \frac{c}{\|g\|_2} g.$$

This preserves the gradient direction while limiting its magnitude, leading to more stable optimisation. Gradient clipping has now become a trick used for training other kinds of neural networks as well.

Next we consider a widely adopted variant on the simple RNN, known as the LSTM.

7.6 Long Short Term Memory Networks (LSTMs)

LSTMs [6] were among the first recurrent architectures capable of reliably learning long-range temporal dependencies before the advent of Transformers. A particularly influential deployment of LSTMs was Google's early neural machine translation system [7], which used LSTM-based sequence-to-sequence models at large scale.

LSTMs use the concept of gates and *internal memory* and *candidate* states to control how sequence data is processed, capturing both long and short range dependencies. One of the key innovations of LSTMs is that they automatically learn from the input sequence when to focus on short or long term dependence, a capability that is lacking in a simple RNN. The improved performance of LSTMs comes at the cost of increased parametric complexity and computational expense (see Exercise 7.6).

Before discussing the structure of an LSTM, we will define some terminology:

- An *unit* or *cell* defines the process by which $h^{(t)}$ is converted to $h^{(t+1)}$, e.g. (7.1), but this can be much more complex as we will see later on.
- A *gate* modulates or controls the flow of information hidden state and input information through a switch. A typical example of a gate is a sigmoid function.
- A *state* is just another word for a model variable, usually indexed by time or iteration number.

We will use the notation $c^{(t)}$ to refer to a candidate state at time t and $\tilde{h}^{(t)}$ to refer to an internal memory state. In a basic LSTM, there are usually three gates, the input gate $g_i^{(t)}$, the forget gate

$g_f^{(t)}$ and the output gate $g_o^{(t)}$, given by

$$g_i^{(t)} = \sigma(W_i h^{(t-1)} + U_i x^{(t)} + b_i) \in \mathbb{R}^{m \times 1} \quad (7.24)$$

$$g_f^{(t)} = \sigma(W_f h^{(t-1)} + U_f x^{(t)} + b_f) \in \mathbb{R}^{m \times 1} \quad (7.25)$$

$$g_o^{(t)} = \sigma(W_o h^{(t-1)} + U_o x^{(t)} + b_o) \in \mathbb{R}^{m \times 1} \quad (7.26)$$

where σ denotes the componentwise sigmoid activation function. Each gate has its own set of weights and biases (notice how the subscript differs), which then increases the parametric complexity of the NN, but also its expressivity. The role of these gates at any given time t are as follows

- **Forget gate:** Controls how information is discarded from the internal memory state at the previous time $\tilde{h}^{(t-1)}$, given current input $x^{(t)}$.
- **Input gate:** Controls how ‘new’ information in the candidate state $c^{(t)}$ is stored in the internal memory state at the current time $\tilde{h}^{(t)}$.
- **Output gate:** Controls which information is output to the hidden state $h^{(t)}$.

The key idea is that the LSTM learns when information should be retained and when it should be overwritten through the gates. These gating functions take a state and multiply each of its components with a number between 0 and 1, crudely speaking, it says what percentage of the information should be let through. If the forget gate $g_f^{(t)}$ is close to zero and the input gate $g_i^{(t)}$ is close to one, the previous memory is largely discarded and replaced by new information from the candidate state. This causes the model to focus on more recent inputs and therefore emphasises short-range dependencies. Conversely, if the forget gate is close to one and the input gate is close to zero, the internal memory state is propagated almost unchanged through time. In this regime, information can persist over many timesteps, allowing the network to capture long-range dependencies.

The presence of these gates means that LSTMs have direct mechanisms of forgetting, remembering and outputting happening simultaneously. Mathematically, this is done by the following operations for the candidate $c^{(t)}$, internal memory $\tilde{h}^{(t)}$ and hidden states $h^{(t)}$ as

$$c^{(t)} = \phi(W_c h^{(t-1)} + U_c x^{(t)} + b_c) \in \mathbb{R}^{m \times 1} \quad (7.27)$$

$$\tilde{h}^{(t)} = g_f^{(t)} \odot \tilde{h}^{(t-1)} + g_i^{(t)} \odot c^{(t)} \in \mathbb{R}^{m \times 1} \quad (7.28)$$

$$h^{(t)} = g_o^{(t)} \odot \psi(\tilde{h}^{(t)}) \in \mathbb{R}^{m \times 1} \quad (7.29)$$

where $\tilde{h}^{(t)}, c^{(t)} \in \mathbb{R}^m$. The function ψ plays the role of a non-linear activation, similar to ϕ . A single step of an LSTM then involves the transformation

$$(x^{(t)}, h^{(t-1)}, \tilde{h}^{(t-1)}) \mapsto (h^{(t)}, \tilde{h}^{(t)}).$$

The candidate state $c^{(t)}$ can be interpreted as the proposed new content at time t and the input gate $g_i^{(t)}$ determines how much of the candidate state is written into the internal memory state. Notice

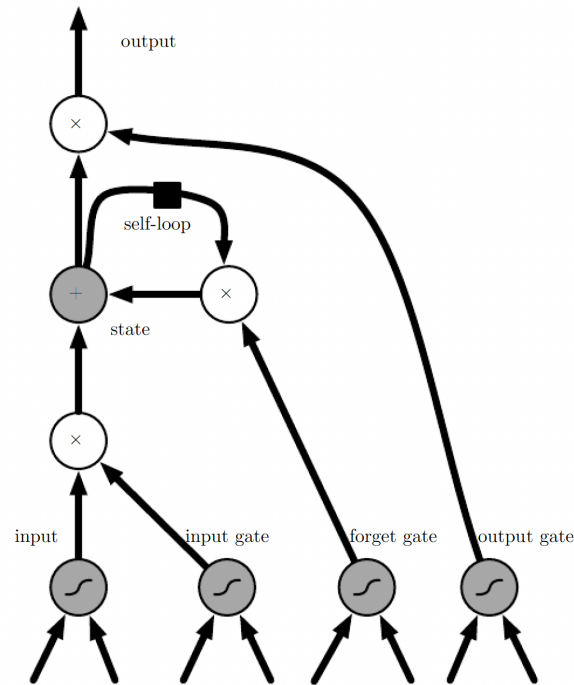


Figure 7.2: Schematic of an LSTM cell (from [3])

also that the candidate state $c^{(t)}$ takes the same form as (7.1) in the simple RNN, but instead of this defining $h^{(t)}$, it goes through 2 further processing steps involving the gating functions, $g_f^{(t)}, g_i^{(t)}, g_o^{(t)}$. A schematic of a simple LSTM is shown in Figure 7.2.

Another useful way to interpret an LSTM is that it separates memory storage from memory access. The internal memory state $\tilde{h}^{(t)}$ acts as a long-term storage mechanism, while the hidden state $h^{(t)}$ is the part of the memory that is made available for immediate computation, such as producing an output or influencing the next timestep. The forget and input gates $g_f^{(t)}$ and $g_i^{(t)}$ respectively determine how information is written to memory, while the output gate $g_o^{(t)}$ determines how information is read from memory. This separation of responsibilities makes it different to a simple RNN where storage, processing and output generation all occur through the same hidden state.

Remark 7.6.1

The forget gate $g_f^{(t)}$ is sometimes referred to as the *conditional self-loop weight*. To see why, consider the internal memory update (7.28). The term $g_f^{(t)} \odot \tilde{h}^{(t-1)}$ acts as a self-loop from the previous memory state to the current memory state. In a classical dynamical system this self-loop would typically have a fixed coefficient. In contrast, the coefficient $g_f^{(t)}$ is computed from the current input and hidden state and therefore **varies over time**. For this reason it is called *conditional*: the amount of memory retention depends on the current context, i.e. current input $x^{(t)}$ and previous hidden state $h^{(t-1)}$. When $g_f^{(t)}$ remains close to one over many timesteps, information stored in the internal memory state can persist almost unchanged, allowing events from the distant past to influence future predictions. The input gate $g_i^{(t)}$ is called conditional for similar reasons, although in reference to how much new information is let through.

Exercise 7.6.1

Notice that the activation functions for the gates (7.24)-(7.26) are sigmoid functions. The activation functions ψ, ϕ in the candidate and hidden state equations are typically taken to be tanh. Why is this the case?

Exercise 7.6.2

What are the total number of (scalar-valued) parameters to be trained in a simple RNN (7.1)-(7.2) compared to the basic LSTM described above?

Exercise 7.6.3

Which of the three gates is the most important mechanism for capturing long range vs short range dependencies?

LSTMs are trained using the same backpropagation through time technique used for RNNs. However, its structure, specifically the introduction of the internal memory state, allows it to avoid the vanishing/exploding gradient issues present with BPTT applied to RNNs.

7.7 Backpropagation through time for LSTMs

Like standard recurrent neural networks, Long Short-Term Memory (LSTM) networks are trained using Backpropagation Through Time. The presence of multiple gates and an internal memory state complicates the chain rule calculations compared to the simple RNN. Nevertheless, the basic principle remains unchanged: first one performs a forward pass through time, then computes time-synchronous partial derivatives of the loss, then propagates total derivatives backward through

time. Once again, consider the case of a single data point and a loss function

$$\mathcal{L} = \sum_{t=1}^T \ell^{(t)}.$$

We will look at the construction for $\frac{d\mathcal{L}}{d(W_f)_{ij}}$, as the pattern is the same for the other parameters. Define the shorthand notation for the input to the activation in the forget gate,

$$a_f^{(t)} := W_f h^{(t-1)} + U_f x^{(t)} + b_f \quad (7.30)$$

We can reuse the same BPTT structure as in the simple RNN case. Recall that for a simple RNN, after introducing the pre-activation

$$a^{(k)} := W h^{(k-1)} + U x^{(k)} + b,$$

the gradient with respect to a recurrent weight in (7.17) was written with $\Delta_a^{(k)} := \frac{d\mathcal{L}}{da^{(k)}} = \Delta_h^{(k)} \odot \dot{\phi}(a^{(k)})$ as

$$\frac{d\mathcal{L}}{dW_{ij}} = \sum_{k=1}^T \left(\frac{\partial a^{(k)}}{\partial W_{ij}} \right)^\top (\Delta_a^{(k)})_i.$$

where the sum is evaluated backwards from T . The forget-gate calculation in an LSTM has exactly the same structure. Define the total forget-gate pre-activation error $\Delta_{a_f}^{(k)} := \frac{d\mathcal{L}}{da_f^{(k)}}$, the LSTM analogue of (7.13). Then

$$\frac{d\mathcal{L}}{d(W_f)_{ij}} = \sum_{k=1}^T \left(\frac{\partial a_f^{(k)}}{\partial (W_f)_{ij}} \right)^\top \Delta_{a_f}^{(k)} = \sum_{k=1}^T h_j^{(k-1)} (\Delta_{a_f}^{(k)})_i.$$

The main difference to BPTT for RNNs is in the calculation of $\Delta_{a_f}^{(k)}$. For the LSTM forget gate, the corresponding chain is

$$a_f^{(k)} \longrightarrow g_f^{(k)} \longrightarrow \tilde{h}^{(k)} \longrightarrow \mathcal{L}.$$

Using the chain rule and the Jacobian convention from Section 7.3.1, we obtain

$$\Delta_{a_f}^{(k)} = \frac{\partial g_f^{(k)}}{\partial a_f^{(k)}} \frac{\partial \tilde{h}^{(k)}}{\partial g_f^{(k)}} \Delta_{\tilde{h}}^{(k)}.$$

where $\Delta_{\tilde{h}}^{(k)} := \frac{\partial \mathcal{L}}{\partial \tilde{h}^{(k)}}$ is the total memory-state error at time k , containing both the local contribution from $\ell^{(k)}$ and all future contributions from later timesteps. Straightforwardly, since $g_f^{(k)} = \sigma(a_f^{(k)})$,

$$\frac{\partial \tilde{h}^{(k)}}{\partial g_f^{(k)}} = \text{diag}(\tilde{h}^{(k-1)}); \quad \frac{\partial g_f^{(k)}}{\partial a_f^{(k)}} = \text{diag}(\dot{\sigma}(a_f^{(k)})).$$

Substituting the two diagonal Jacobians gives

$$\Delta_{a_f}^{(k)} = \text{diag}\left(\dot{\sigma}(a_f^{(k)})\right) \text{diag}\left(\tilde{h}^{(k-1)}\right) \Delta_{\tilde{h}}^{(k)} \quad (7.31)$$

$$= \left(\Delta_{\tilde{h}}^{(k)} \odot \tilde{h}^{(k-1)}\right) \odot \dot{\sigma}\left(a_f^{(k)}\right). \quad (7.32)$$

It remains to compute the total memory-state error $\Delta_{\tilde{h}}^{(k)}$. As in the simple RNN derivation leading to (7.12), this total derivative contains a same-time contribution and a future contribution. The same-time contribution comes from the dependence of $h^{(k)}$ on $\tilde{h}^{(k)}$, while the future contribution comes from the dependence of $\tilde{h}^{(k+1)}$ on $\tilde{h}^{(k)}$.

Exercise 7.7.1

Using the same calculations as in (7.10)-(7.12), show that

$$\Delta_{\tilde{h}}^{(k)} = \frac{\partial h^{(k)}}{\partial \tilde{h}^{(k)}} \delta_h^{(k)} + \frac{\partial \tilde{h}^{(k+1)}}{\partial \tilde{h}^{(k)}} \Delta_{\tilde{h}}^{(k+1)}. \quad (7.33)$$

Finally, for the two Jacobians in (7.33), recall that $h^{(k)} = g_o^{(k)} \odot \psi(\tilde{h}^{(k)})$, so then again by chain rule,

$$\frac{\partial h^{(k)}}{\partial \tilde{h}^{(k)}} = \text{diag}\left(g_o^{(k)} \odot \dot{\psi}(\tilde{h}^{(k)})\right).$$

Hence

$$\frac{\partial h^{(k)}}{\partial \tilde{h}^{(k)}} \delta_h^{(k)} = \delta_h^{(k)} \odot g_o^{(k)} \odot \dot{\psi}(\tilde{h}^{(k)}).$$

Similarly, recall $\tilde{h}^{(k+1)} = g_f^{(k+1)} \odot \tilde{h}^{(k)} + g_i^{(k+1)} \odot c^{(k+1)}$, so then

$$\frac{\partial \tilde{h}^{(k+1)}}{\partial \tilde{h}^{(k)}} = \text{diag}\left(g_f^{(k+1)}\right). \quad (7.34)$$

Therefore

$$\frac{\partial \tilde{h}^{(k+1)}}{\partial \tilde{h}^{(k)}} \Delta_{\tilde{h}}^{(k+1)} = \Delta_{\tilde{h}}^{(k+1)} \odot g_f^{(k+1)}.$$

Substituting these two expressions into (7.33) gives

$$\Delta_{\tilde{h}}^{(k)} = \delta_h^{(k)} \odot g_o^{(k)} \odot \dot{\psi}(\tilde{h}^{(k)}) + \Delta_{\tilde{h}}^{(k+1)} \odot g_f^{(k+1)}. \quad (7.35)$$

Putting everything together yields

$$\frac{d\mathcal{L}}{d(W_f)_{ij}} = \sum_{k=1}^T h_j^{(k-1)} \left(\Delta_{\tilde{h}}^{(k)} \odot \tilde{h}^{(k-1)} \odot \dot{\sigma}(a_f^{(k)}) \right)_i.$$

where $\Delta_{\tilde{h}}^{(k)}$ is given by 7.35 and $\delta_h^{(k)}$ depends on the form of the output, and once again the sum is evaluated backwards in time.

When the forget gate remains close to one, the future memory gradient is passed backward with little attenuation. This is known as the *constant error carousel* in [6] and is the main reason LSTMs are better able than simple RNNs to learn long-range temporal dependencies, which we'll see in more detail next.

7.8 How do LSTMs mitigate vanishing gradients?

One of the main reasons LSTMs were introduced was to mitigate the vanishing gradient issue of simple RNNs. We saw that in the LSTM above, the internal memory state $\tilde{h}^{(t)}$ was one of the main innovations that avoids this problem. Unlike a simple RNN, an LSTM contains multiple pathways through which gradient information can propagate backward through time. In particular, gradients can flow both through the hidden state pathway and through the internal memory pathway as in (7.35). During backpropagation, the total gradient arriving at a memory state is therefore a sum of contributions from multiple routes. Therefore,

- The internal memory state thus acts as a dedicated channel for transporting information across **long temporal distances**,
- The hidden state continues to perform **short-term processing** and output generation.

Just as we did in Section 7.3.3, we'll consider an alternative view on $\Delta_{\tilde{h}}^{(k)}$ to the previous section, i.e.

$$\Delta_{\tilde{h}}^{(k)} = \sum_{l=k}^T \frac{\partial \tilde{h}^{(l)}}{\partial \tilde{h}^{(k)}} \delta_{\tilde{h}}^{(l)}.$$

Again, consider the summand term which once again has a product structure after applying chain rule and using (7.34),

$$\frac{\partial \tilde{h}^{(l)}}{\partial \tilde{h}^{(k)}} = \prod_{j=k+1}^l \frac{\partial \tilde{h}^{(j)}}{\partial \tilde{h}^{(j-1)}} = \prod_{j=k+1}^l \text{diag}(g_f^{(j)}). \quad (7.36)$$

Compare this to the product term arising in a simple RNN,

$$\frac{\partial h^{(l)}}{\partial h^{(k)}} = \prod_{j=k+1}^l W^\top \text{diag}(\dot{\phi}(a^{(j)})),$$

So BPTT applied to LSTMs still relies on a product-gradient structure, with the key difference being that there is now a function (whose parameters are learned) that can avoid vanishing/exploding gradients separately to the other weight parameters (e.g. W_c). Most importantly, the activation derivative term is avoided as well.

Consider the extreme case where $g_f^{(j)} \approx \mathbf{1}$ for $j = k + 1, \dots, l$, then

$$\frac{\partial \tilde{h}^{(l)}}{\partial \tilde{h}^{(k)}} = \prod_{j=k+1}^l \text{diag}(g_f^{(j)}) \approx I,$$

which as we saw at the end of Section 7.3.3 is extremely difficult for RNNs to achieve. This also motivates the use of the sigmoid as the activation function for the forget gate, rather than some other activation function that does not map to $(0, 1)$ such as the ReLU.

LSTMs Do Not Completely Eliminate Gradient Problems.

Although LSTMs substantially mitigate the vanishing gradient problem, they do not eliminate all optimisation difficulties. The constant error carousel provides a nearly linear path through the internal memory state, so gradients can be preserved when the forget gates remain close to one. However, this preservation is conditional: if the forget gates are consistently below one, the memory-path gradient is still multiplied by a product of gate values and can decay over long time intervals.

7.9 LSTM implementation aspects and limitations

Although LSTMs substantially improve upon simple RNNs, their successful deployment depends on several practical implementation choices. In addition to gradient clipping, another important implementation aspect is *Forget gate bias initialisation*. Since the sigmoid function is monotone increasing, a positive forget-gate bias therefore causes the network to initially retain a large proportion of its memory. This encourages information to persist through time during the early stages of training. Empirically, this often leads to more stable optimisation because the network begins by remembering most information and subsequently learns which information should be discarded, rather than prematurely forgetting potentially useful signals. We will see an example of the usefulness of this in the Lab.

Some limitations of LSTMs are:

- **Computational expense.** The improved memory capabilities of LSTMs come at the cost of increased computational and memory requirements (see Exercise 7.6).
- **Vanishing gradient issue is not eliminated.** As we saw in Section 7.8, if the forget gate repeatedly takes values significantly below one, memory will still gradually decay. Although LSTMs are much better at modelling long-range dependencies than simple RNNs, they do not provide perfect memory capabilities over arbitrarily long sequences.
- **Sequential computation.** The recurrent updates in LSTMs (and RNNs) must be evaluated sequentially in time. In particular, the states at time t cannot be computed until the states at time $t - 1$ are available. This means that the

$$h^{(1)}, h^{(2)}, \dots, h^{(T)}$$

cannot be fully parallelised across the sequence. This sequential dependency affects both training and inference. During training, forward propagation and Backpropagation Through Time must process the sequence one step at a time. During inference, generating outputs for long sequences can become computationally expensive because each new state depends on all previous computations. As datasets and models become larger, this lack of parallelism becomes a significant practical limitation.

The final limitation was one of the major issues addressed by Transformers. In a Transformer, any two tokens (elements of the data sequence) can interact directly through what's known as a *self-attention* layer. For a concise introduction to Transformers, see <https://arxiv.org/abs/2304.10557>.

7.10 Tutorial exercises

Exercise 1. Complete all exercises within the lecture notes.

Exercise 2. Consider a highly simplified linear RNN in 1D with a single data point (x, y) with $x, y \in \mathbb{R}$,

$$\begin{aligned} h^{(t)} &= wh^{(t-1)} + ux^{(t)}, \quad h^{(0)} = 0, \quad x^{(1)} = x \\ x^{(t)} &= 0 \quad \forall \quad t = 2, 3, \dots, T \\ \hat{y} &= h_T \end{aligned}$$

for some $T > 0$. The loss function is given by

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$$

- i. Calculate the optimal w (i.e. analytic solution) using standard methods.
- ii. Calculate the solution for w using BPTT.
- iii. Calculate the limit of $\frac{d\mathcal{L}}{dw}$ as $T \rightarrow \infty$. What does this suggest about the nature of the derivative for $|w| < 1$ and $|w| > 1$, and therefore the success of a gradient descent based method for learning long range dependence?

Exercise 3. Consider data generated by the one-dimensional nonlinear autoregressive model with known inputs

$$\begin{aligned} s^{(t)} &= a_{\star}s^{(t-1)} + \beta_{\star}x^{(t)}, \quad s^{(0)} = 0, \\ y^{(T)} &= \psi(s^{(T)}), \end{aligned}$$

where $a_{\star}, \beta_{\star}, x^{(t)} \in \mathbb{R}$, and ψ is a differentiable nonlinear function. Suppose we observe the input sequence $(x^{(1)}, \dots, x^{(T)})$ and want to predict the final output $y^{(T)}$. We train the one-dimensional nonlinear RNN

$$\begin{aligned} h^{(t)} &= \phi(a^{(t)}), \\ a^{(t)} &= wh^{(t-1)} + ux^{(t)} + b, \quad h^{(0)} = 0, \\ \hat{y}^{(T)} &= h^{(T)}, \end{aligned}$$

using the squared loss

$$\mathcal{L} = \frac{1}{2} \left(\hat{y}^{(T)} - y^{(T)} \right)^2.$$

- i. Show that the true output is

$$y^{(T)} = \psi \left(\beta_{\star} \sum_{j=1}^T a_{\star}^{T-j} x^{(j)} \right).$$

- ii. What is the main structural difference between the RNN and the true data generating process?
- iii. Calculate the BPTT gradients $\frac{d\mathcal{L}}{dw}, \frac{d\mathcal{L}}{du}, \frac{d\mathcal{L}}{db}$ in the form of (7.19).
- iv. Suppose there exists a constant $0 < \rho_\phi < 1$ such that

$$|\dot{\phi}(a^{(j)})| \leq \rho_\phi \quad \text{for all } j$$

Show that

$$\left| \frac{\partial h^{(T)}}{\partial h^{(k)}} \right| \leq (\rho_\phi |w|)^{T-k}.$$

Explain why this makes learning long-range dependence difficult when $a_\star \approx 1$.

Exercise 4. Consider the same data-generating process as in Exercise 7.10:

$$\begin{aligned} s^{(t)} &= a_\star s^{(t-1)} + \beta_\star x^{(t)}, & s^{(0)} &= 0, \\ y^{(T)} &= \psi(s^{(T)}). \end{aligned}$$

We now train the following simplified one-dimensional LSTM:

$$\begin{aligned} c^{(t)} &= vx^{(t)}, \\ \tilde{h}^{(t)} &= g_f^{(t)} \tilde{h}^{(t-1)} + g_i^{(t)} c^{(t)}, & \tilde{h}^{(0)} &= 0, \\ \hat{y}^{(T)} &= \psi(\tilde{h}^{(T)}). \end{aligned}$$

Here $g_f^{(t)}$ is the forget gate, $g_i^{(t)}$ is the input gate, and $\tilde{h}^{(t)}$ is the internal memory state. For this exercise, treat $\{g_f^{(t)}, g_i^{(t)}\}_{t=1}^T$ as scalar trainable quantities, with $0 \leq g_f^{(t)}, g_i^{(t)} \leq 1$.

The loss is

$$\mathcal{L} = \frac{1}{2} \left(\hat{y}^{(T)} - y^{(T)} \right)^2.$$

- i. Show that

$$\tilde{h}^{(T)} = v \sum_{j=1}^T g_i^{(j)} \left(\prod_{r=j+1}^T g_f^{(r)} \right) x^{(j)},$$

where an empty product is interpreted as 1.

- ii. Find an analytic choice of v , $g_i^{(j)}$, and $g_f^{(r)}$ that matches the true model for all input sequences, if such a choice is possible.
- iii. Calculate the BPTT gradients with respect to v , $g_i^{(t)}$, and $g_f^{(t)}$.
- iv. Using your response to (iii), identify the term that allows LSTMs to mitigate vanishing or exploding gradients.
- v. By making use of the answers in (ii) and (iii), explain why LSTMs can mitigate, but not entirely remove the vanishing gradient issue.

References

- [1] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
- [2] Anton Maximilian Schafer and Hans Georg Zimmermann. Recurrent Neural Networks Are Universal Approximators. In *Kollias, S.D., Stafylopatis, A., Duch, W., Oja, E. (eds) Artificial Neural Networks – ICANN 2006.*, Lecture Notes in Computer Science. Springer, 2006.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [4] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.
- [5] Benoit Lique, Sarat Moka, and Yoni Nazarathy. *Mathematical Engineering of Deep Learning*. CRC Press, 2024.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [7] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

A [Non-examinable] Proof of Universal Approximation Theorem

Proof: The proof in [2] largely relies on UAT for FNNs, Theorem 2.3.1 from Week 2. We’ll use the shorthand notation

$$\tilde{f}(h, x) := \phi(Wh + Ux + b).$$

By a direct application of Theorem 2.3.1 (Week 2), there exists W, U, b and $\delta > 0$ such that

$$\sup_{s^{(0)}, s^{(t-1)}, x^{(t)} \in \Omega} \|g(s^{(t-1)}, x^{(t)}) - \tilde{f}(s^{(t-1)}, x^{(t)})\| < \delta \quad \text{for all } t = 1, 2, \dots, T. \quad (7.37)$$

where Ω is a compact set containing all possible $s^{(t)}, x^{(t)}, h^{(t)}$ for all $t = 1, 2, \dots, T$. Compactness is important because the feedforward universal approximation theorem gives uniform approximation on compact domains. It also ensures that continuous maps such as g and $\tilde{f}$ are uniformly continuous on the relevant region, which is needed to control how approximation errors propagate through

time. We must now extend this result so that the RNN is acting on $h^{(t)}$, so that we can prove approximation capabilities for the iterated map. The standard trick here is to use *telescoping*,

$$\begin{aligned} \|s^{(t)} - h^{(t)}\| &= \|g(s^{(t-1)}, x^{(t)}) - \tilde{f}(h^{(t-1)}, x^{(t)})\| \\ &= \|g(s^{(t-1)}, x^{(t)}) - \tilde{f}(s^{(t-1)}, x^{(t)}) + \tilde{f}(s^{(t-1)}, x^{(t)}) - \tilde{f}(h^{(t-1)}, x^{(t)})\| \\ &\leq \|g(s^{(t-1)}, x^{(t)}) - \tilde{f}(s^{(t-1)}, x^{(t)})\| + \|\tilde{f}(s^{(t-1)}, x^{(t)}) - \tilde{f}(h^{(t-1)}, x^{(t)})\| \end{aligned}$$

where in the last line we've used the triangle inequality. The first term is easily bounded using (7.37). For the second term, we will make use of *uniform continuity* of $\tilde{f}$. Recall that a function $\tilde{f}$ is uniformly continuous if for every $\varepsilon > 0$, there exists a $\delta > 0$ such that for any $h, \bar{h} \in \mathbb{R}^m$,

$$\|h - \bar{h}\| < \delta \implies \|\tilde{f}(h, x) - \tilde{f}(\bar{h}, x)\| < \varepsilon \quad (7.38)$$

uniformly for all $x \in \mathbb{R}^d$. *Uniformly* means that the same δ works throughout the whole compact domain, rather than depending on a particular value of h , $\bar{h}$, or x . Also recall that the sigmoid function satisfies the properties of uniform continuity.

We'll now make use of an *induction* argument to show that the error stays controlled when iterating over time. For $t = 1$, since we assume $s^{(0)} = h^{(0)}$, we clearly have that $\|\tilde{f}(s^{(0)}, x^{(1)}) - \tilde{f}(h^{(0)}, x^{(1)})\| = 0$ so that

$$\|s^{(1)} - h^{(1)}\| \leq \delta. \quad (7.39)$$

Then for $t = 2$, (7.39) and uniform continuity of $\tilde{f}$ gives

$$\|\tilde{f}(s^{(1)}, x^{(2)}) - \tilde{f}(h^{(1)}, x^{(2)})\| < \varepsilon_2$$

for some $\varepsilon_2 > 0$ so that

$$\|s^{(2)} - h^{(2)}\| < \delta + \varepsilon_2.$$

Then for $t = 3$, by the same arguments we have

$$\|s^{(3)} - h^{(3)}\| < \delta + \varepsilon_3.$$

for some $\varepsilon_3 > 0$. Then by induction, we have that $\|s^{(t)} - h^{(t)}\| < \varepsilon$ where $0 < \varepsilon < \infty$ represents some error tolerance.

It remains to pass from hidden-state approximation to output approximation. Choosing $V = Q$ and $c = r$, we obtain

$$\begin{aligned} \|z^{(t)} - \hat{y}^{(t)}\|_2 &= \|Qs^{(t)} + r - (Qh^{(t)} + r)\| \\ &= \|Q(s^{(t)} - h^{(t)})\| \\ &\leq \|Q\|_2 \|s^{(t)} - h^{(t)}\|. \end{aligned}$$

■